

Reverse Engineering Foundations: ELF, PE & Static Analysis

Breaking Linux Binaries
from Scratch

Presenter: Md. Tanjim Mahmud Tuhin
Date: 08/08/2026



Presented to
EWUCSC



Before we begin: a story

1997

A locked vending machine.

No key. No manual. A student watched which coin it accepted, listened to the mechanism, and mapped it from the outside in.

Software is the same. You cannot read the source, but the program still tells you everything if you know how to listen.

```
01111111 ELF 02 01 01
```

↓ reverse

```
if (password == secret)
```

Every program reveals its secrets — if you know where to look.



Same skill. Real stakes.

● Malware Analysis

Understand what an unknown binary really does.

DFIR

↘ Vulnerability Research

Find bugs without source code.

OFFENSIVE

▣ License & Logic Bypass

Understand software protections.

SOFTWARE SECURITY

◇ CTF Competitions

Build the skill by solving small challenges.

PRACTICE

Today you will learn **how all of these begin.**

What you will walk away with

- 01 Recognize the structure of a Linux ELF binary.

- 02 Inspect a file without running it using basic tools.

- 03 Understand registers, arguments, comparisons, and jumps.

- 04 Build a small checker, then reverse your own binary.

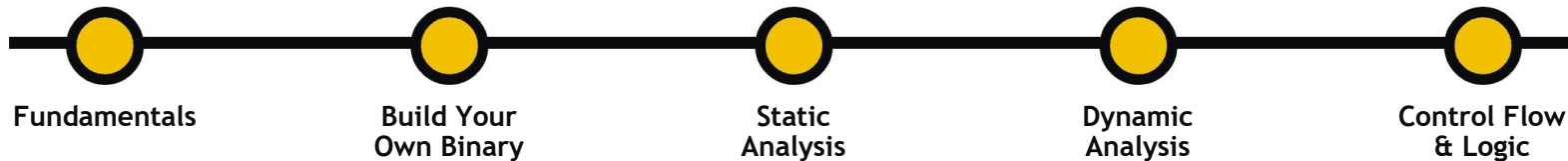
HANDS-ON LABS

TryHackMe “Reversing ELF”

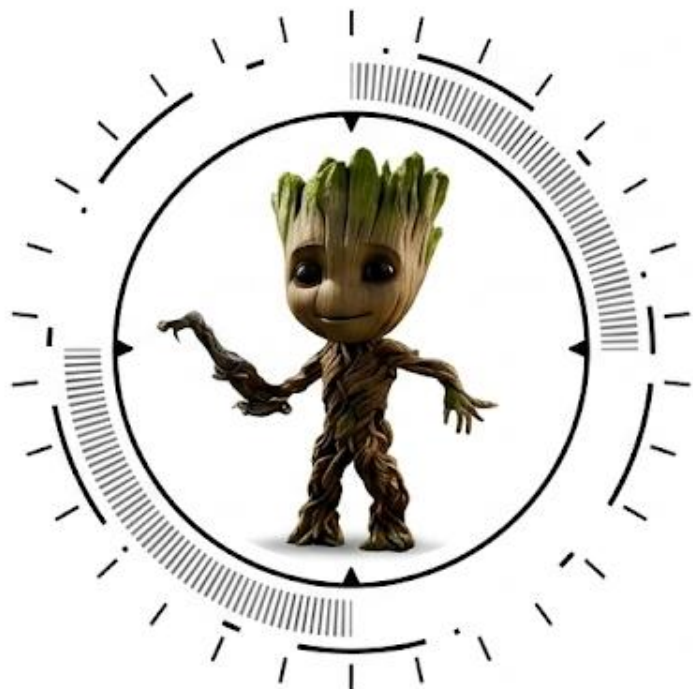
Every exercise turns one idea into something you can test.

The goal: learn a repeatable way to ask a binary better questions.

The route from source to secrets



Start with the shape of the binary. Then learn how it thinks.



WHO AM I

Tanjim M. Tuhin

PROFESSIONAL EXPERIENCE

- Govt. Project — SOC Analyst & Forensics Investigator

CERTIFICATIONS

- CEH v13 (EC-Council Certified Ethical Hacker)
- CHFI v11 (EC-Council Computer Hacking Forensic Investigator)
- CSA (EC-Council Certified SOC Analyst)
- ECIH (EC-Council Certified Incident Handler)

PREVIOUS EXPERIENCE

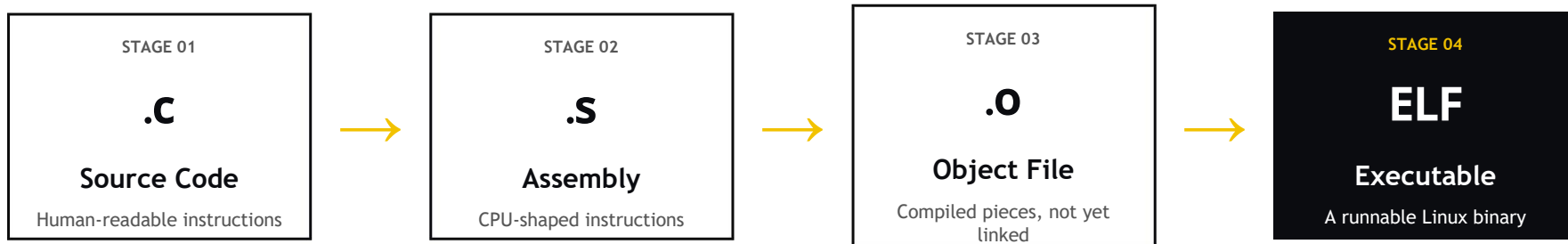
- RedData — Network Engineer(RnD)
- RBD — Unity Developer
- Cybernetics — Robotics Engineer(Hardware)

Vision

- 'Learn together to build a skilled community.'

00

From code to machine



Each step removes human context – and leaves machine clues.

ELF is the binary's container

MAGIC BYTES

7F 45 4C 46

45 4C 46 → E L F

The first four bytes identify the file format.

Executable and Linkable Format

ELF is the standard Linux binary container. It organizes the instructions, data, and metadata a program needs to run.

Why begin here?

Before reading assembly, confirm what kind of file you are holding. The header gives you the first reliable clue.

file ./binary → ELF 64-bit executable

Read the header first. It tells you what kind of puzzle you are holding.

Inside an ELF: four useful rooms

.text	The instructions the CPU executes
.rodata	Constants and strings — where clues often hide
.data	Initialized variables
.bss	Storage reserved for uninitialized data

A SIMPLE ANALOGY

Think of sections as labeled rooms.

Each room stores a different kind of information. The labels tell you where to look before you read the details.

.rodata is the first room beginners inspect.

Start with the room that stores readable constants.

Registers are the CPU's pockets

The CPU keeps small, fast values close at hand.

RAX / EAX / AL	Return values and comparisons
RDI	First function argument
RSI	Second function argument
RSP / RBP	Stack pointers

ONE POCKET, SMALLER SLICES

RAX

64-bit register

EAX

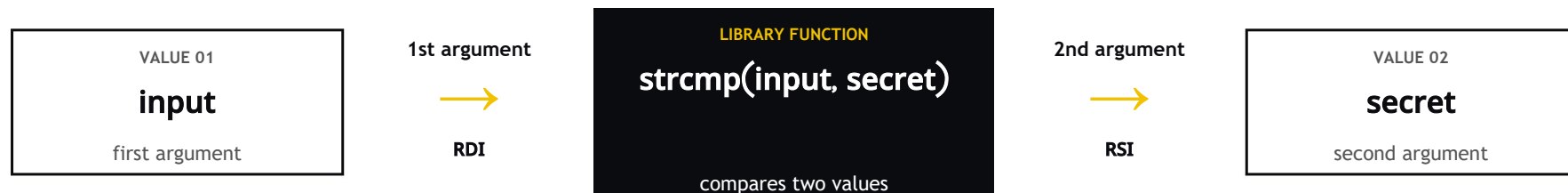
lower 32 bits

AL

lower 8 bits

The name tells you how much of the pocket you are reading.

Functions receive arguments in registers



On common 64-bit Linux systems, the first function arguments arrive in predictable registers.

If you can read RDI and RSI, you can read the password.

Compare, then choose

cmp – compare two values

test – check bits or zero

je – jump when equal

jne – jump when different

TERMINAL BLOCK / ASSEMBLY

```
cmp eax, 0x1
```

```
jne fail
```

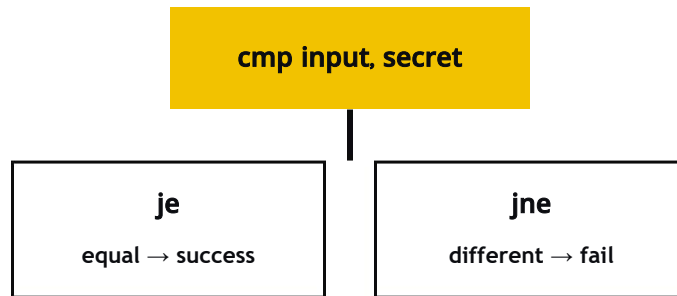
```
call success
```

```
fail:
```

```
call retry
```

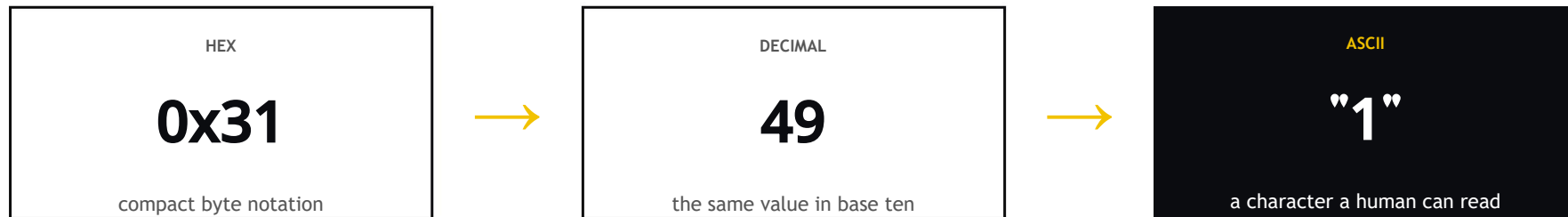
The compare sets a condition. The jump chooses which path runs next.

The decision point



A password check is usually a compare followed by a choice.

Numbers become characters



SIGNED vs UNSIGNED

The same bits can represent a positive number or a negative number, depending on how the program interprets them.

TWO'S COMPLEMENT

A common bit pattern used to store negative integers in binary.

The same bytes can look like numbers, characters, or instructions – context tells you which.

Reversing ELF

From Building to Watching Binaries

```
gcc -o binary source.c
```

```
strings ./binary
```

```
ltrace ./binary
```

```
BUILD → INSPECT → WATCH
```

Write the lock before picking it

C SOURCE / TERMINAL BLOCK

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char input[32];
    scanf("%31s", input);
    if (strcmp(input, "CHANGE_ME") == 0)
        puts("Access granted");
    else puts("Try again");
    return 0;
}
```

```
gcc -o mycrackme mycrackme.c
```

A binary is easier to understand when you know what created it.

01

Write the checker

Start with familiar source code.

02

Compile the checker

Turn the source into an ELF executable.

03

Reverse your own binary

Recognize how the same idea changes shape.

Build the lock first. Then learn how the binary remembers it.

Technique 1 – read without running

Static analysis = inspecting a program without executing it.

file	Identify the format and architecture Tag: IDENTIFY
strings	Print readable sequences embedded in a file Tag: QUICK WIN
rabin2 -z	List strings through the radare2 toolkit Tag: RADARE2
objdump	Display low-level file and instruction details Tag: DISASSEMBLE

Start with the cheapest clue before reaching for the deepest tool.

Lab 0 — your first binary

01 Compile it

Turn the source checker into an executable.

02 Identify it

Use `file` to confirm the binary format.

03 Inspect it

Use `strings` to find readable clues.

04 Run and compare

Compare what the source said with what the binary reveals.

COMMANDS / TERMINAL BLOCK

```
gcc -o mycrackme mycrackme.c
```

```
file ./mycrackme
```

```
strings ./mycrackme
```

```
./mycrackme
```

Observe first. Explain second. Do not guess from one clue.

You just reverse engineered yourself.

When secrets are stored in plain sight

TERMINAL BLOCK / STATIC CLUE

\$ strings ./binary

Usage: ./binary <input>

Access granted

Try again

[printable text only]

Readable constants often live in the read-only data section.

Why does strings work?

Constants can survive compilation.

Messages, labels, and comparison text may remain as printable bytes inside the file.

Output is a clue, not proof.

A string may be unused, decoy text, or only one piece of the program's logic.

Encoded is not encrypted.

Base64 often looks like a long mix of letters, numbers, + /, and optional = padding.

A-Z a-z 0-9 + / =

Start with readable text. Then ask whether the program actually uses it.

LAB 1: CRACKME 1, 2, 3



Run it.

Execute the binary and observe the output.

RUNTIME



strings reveals...

Look for human-readable strings in the binary.

STATIC



strings reveals Base64.

Find the encoded string and decode it.

ENCODING

```
$ ./crackme1  
$ strings crackme2  
$ strings crackme3 | base64 -d
```

"Always try the easy thing first."



When static goes quiet

The Scenario: You run `strings` and `rabin2 -z`, but the output is empty or filled with junk.

Why? The developer didn't store the password as a static string. It's being built character-by-character or decrypted at runtime.

The Solution: If we can't read the secret while it's "sleeping" (on disk), we watch it while it's "awake" (running).

Dynamic Analysis: The art of observing a program's behavior, memory, and interactions during execution.

[NO STRINGS FOUND]

SWITCH TO DYNAMIC





Technique 2: Watching library calls

The Doorway (PLT)

Most programs "borrow" functions like `strcmp` from Linux libraries (libc) rather than writing their own.

The Interception

We can stand at the "doorway" (Procedure Linkage Table) and watch every argument passed to these borrowed functions.

Tool: ltrace

A library call tracer that intercepts and records dynamic library calls as they happen.

```
$ ltrace ./crackme4
...
strcmp("user_input", "          ") = 1
...
```

Note: The second argument is often the secret we are looking for.



LAB 2: CRACKME 4, 5

Crackme4

strings fails, ltrace wins. Intercept the `strcmp` call to find the flag.

LTRACE

Crackme5

Decoys and Lengths. Watch for `strncmp` and ignore decoy `strlen` calls.

FILTERING

```
$ ltrace ./crackme4
$ ltrace ./crackme5
```

"If libc does the comparing, ltrace does the cracking."



The developer fights back

The Scenario

`ltrace` shows nothing. No `strcmp`, no `memcmp` no library calls at all. The program is silent.

The Tactic

The developer wrote their own comparison logic "inline" instead of calling a shared library.

Inline Comparison

The CPU compares each byte of your input directly against hardcoded values in its registers.

The Result

The secret never leaves the program's internal logic. It never crosses the "doorway" where `ltrace` is watching.

BINARY INTERNAL LOGIC

0x46

0x4C

0x41

0x47



LIBC CALL PATH BLOCKED



Technique 3: Read the disassembly

STEP 1: OPEN & DEBUG

```
r2 -d ./binary
```

Load the binary

Load the target file into Radare2 in debug mode to allow runtime inspection.

STEP 2: ANALYZE

```
aaa
```

Run Analysis

Perform full analysis to identify functions, strings, and cross-references.

STEP 3: LIST FUNCTIONS

```
afl
```

Find the Entry Point

Look for `main` or suspicious function names like `sym.check_password`.

STEP 4: DISASSEMBLE

```
pdf @sym.function
```

Read the Code

Print the disassembly of the selected function to see the raw instructions.



Look for the Chain:
Hexadecimal

A sequence of

`cmp al, 0xNN`

instructions is often a password spelled out character-by-character in

LAB 3: CRACKME 6

1 Locate

Find the function where the input is being checked (e.g. `sym.check_password`).

2 Collect

Identify the sequence of `cmp` or `mov` instructions involving hex values.

3 Convert

Take those hex bytes and turn them back into readable ASCII characters.

0x31	0x33	0x33	0x37
------	------	------	------



ASCII CONVERSION

1	3	3	7
---	---	---	---

"When the program won't talk, read its mind."



Technique 4: Follow the logic

Beyond Passwords

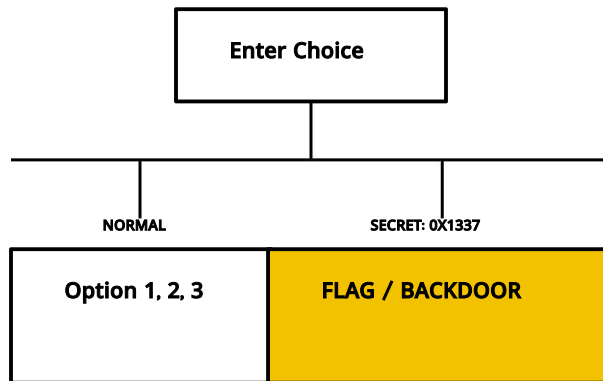
Sometimes the goal isn't a string, but a specific path through the code.

Hidden Paths

Developers often leave "backdoors" or hidden menu options for testing.

Control Flow

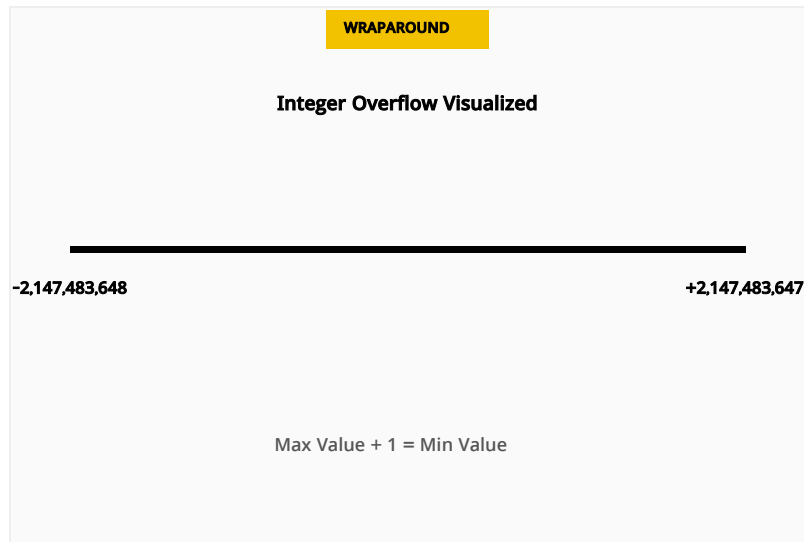
By reading disassembly, we map out every if/else branch (jumps).





When math becomes the lock

- **Integer Overflow:** Occurs when a program stores a value larger than its capacity.
- **The 32-bit Limit:** Signed integers max out at 2,147,483,647.
- **The Wraparound:** Adding 1 to the max value flips it to -2,147,483,648.
- **Logic Flaw:** Programs using `atoi()` for input conversion are prime targets for this bypass.



"THE BUG ISN'T IN THE PASSWORD. IT'S IN THE ARITHMETIC."



Lab 4: Crackme 7 & 8

Crackme 7: Hidden Logic

Find the secret menu

Use `pdf @main` in radare2 to find a branch that isn't jumped to in the program's output. Look for conditional jumps (`je`, `jne`) that lead to unreferenced code blocks.

LOGIC FLOW

Crackme 8: Integer Math

Defeat the comparison

Identify the variable type and its maximum value. Provide a specific input that causes an overflow to bypass a "must be less than" security check.

ARITHMETIC

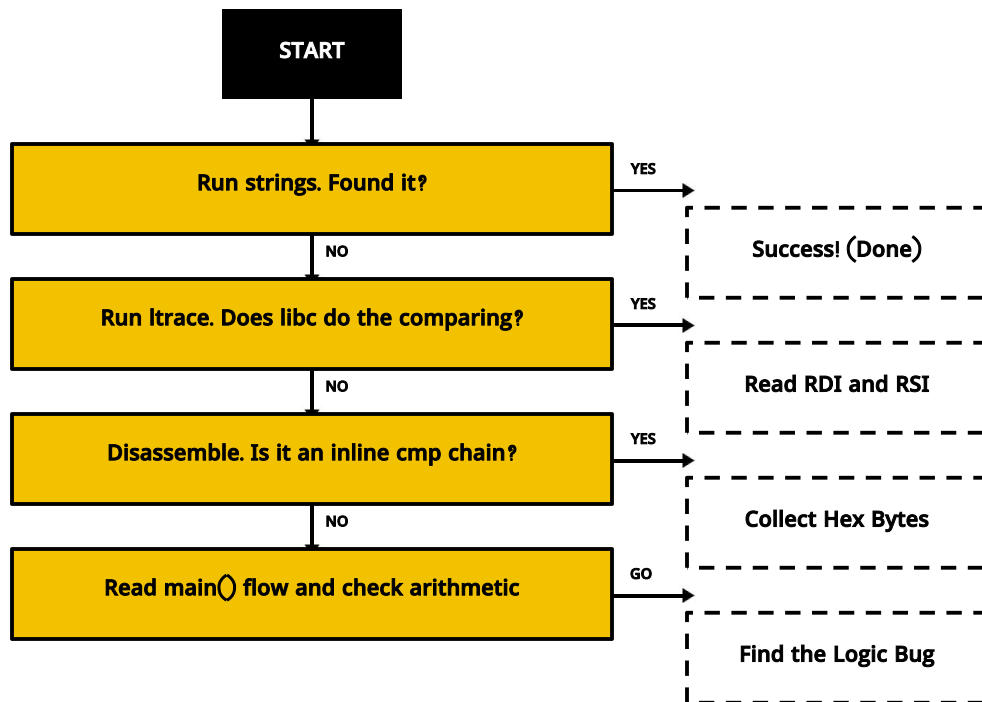
Commands for Lab 4

```
r2 -d ./crackme7
aaa: afl; pdf @main
./crackme8 [large_value]
```

"LOGIC FLOWS WHERE THE CODE GOES — FOLLOW THE BRANCHES."



Your decision tree





Where to go next



x86-64 Assembly

Master the fundamental language of the CPU to understand exactly how instructions manipulate data.



C Internals

Understand how high-level C code is structured in memory, including the stack, heap, and pointers.



Ghidra & Decompilers

Move beyond assembly by using professional tools to translate binary code back into readable pseudo-code.



More CTF Rooms

Practice your skills on TryHackMe, HackTheBox, and PicoCTF to encounter more complex binary challenges.

**STATIC FIRST. DYNAMIC WHEN STATIC GOES QUIET.
DISASSEMBLE WHEN NOTHING ELSE TALKS.**



Need help after class? Join our  **Discord**



<https://hiddeninvestigations.net/community>



Hidden Investigations



Thank You

Learn together, to build a skilled community